DSA Notes in Python

Contents

1 Hash Tables: Dictionaries and Sets **2**
- 1.1 Dictionary . . . 2
- 1.2 Sets . . . 2

2 Arrays **2**
- 2.1 Static Arrays . . . 2
- 2.2 Dynamic Arrays . . . 2
- 2.3 Common Operations on Python Lists . . . 3
- 2.4 Implementation of a Dynamic Array . . . 3

3 Strings **3**
- 3.1 Basics of Strings . . . 3
- 3.2 Common Built-in Methods . . . 3
- 3.3 Character ↔ ASCII Conversion . . . 4
- 3.4 String ↔ list conversion (in-place-style manipulation) 4
- 3.5 String Comparison . . . 4

4 Sorting **4**
- 4.1 Time Complexities . . . 4
- 4.2 Comparison based Algorithms . . . 4
- 4.3 Non-comparison based Algorithms . . . 5

5 Linked Lists **6**
- 5.1 Basic Structure . . . 6
- 5.2 Reversing a Linked List . . . 6
- 5.3 Detecting Cycle . . . 6

6 Queue **6**
- 6.1 Introduction . . . 6
- 6.2 Some important features/functions on `deque` . . . 6
- 6.3 An important note . . . 6
- 6.4 Implementation of Queues using LinkedLists . . . 7
- 6.5 Implementation of Queues using two stacks . . . 7
- 6.6 Implementation of Circular Queue . . . 7

7 Double-ended Queues **7**
- 7.1 Introduction . . . 7
- 7.2 Implementation using Doubly Linked Lists . . . 7

8 Stack **8**
- 8.1 Stack Basics . . . 8
- 8.2 Implementation using LinkedLists . . . 8

9 Heaps / Priority Queues **8**
- 9.1 Basics . . . 8
- 9.2 `heapq` module . . . 8
- 9.3 Implementation of heaps from scratch . . . 8

10 Binary Trees **9**
- 10.1 Binary Trees Basics . . . 9
- 10.2 Binary Trees Traversal . . . 9
- 10.3 Constructing a Binary Tree from preorder and inorder traversals . . . 9
- 10.4 Height of a Binary Tree . . . 10
- 10.5 Number of Nodes in a Binary Tree . . . 10
- 10.6 Is Identical . . . 10
- 10.7 Is Subtree . . . 10
- 10.8 Diameter of a binary tree . . . 10
- 10.9 Check if balanced . . . 10

11 Binary Search Trees **10**
- 11.1 Basics . . . 10
- 11.2 Implementation from Scratch . . . 10
- 11.3 Validate a BST . . . 11
- 11.4 Lowest Common Ancestor (LCA) . . . 11

12 Graphs **11**
- 12.1 Basics . . . 11
- 12.2 Representation . . . 11
- 12.3 Traversals . . . 11
- 12.4 Common Algorithms . . . 12
- 12.5 Shortest Paths . . . 12
- 12.6 Minimum Spanning Tree . . . 13
- 12.7 Minimum Height Trees . . . 14

13 Bit Manipulation **14**
- 13.1 Basics . . . 14
- 13.2 Common Algorithms . . . 14

1 Hash Tables: Dictionaries and Sets

1.1 Dictionary

```
d = {'a': 1, 'b': 2}
```

Common Operations

Operation	Syntax	Average case	Worst case
Access value	`d['a']` or `d.get('a')`	$O(1)$	$O(n)$
Insert / update	`d['c'] = 3`	$O(1)$	$O(n)$
Delete	`del d['a']` or `d.pop('a')`	$O(1)$	$O(n)$
Check key exists	`'a' in d`	$O(1)$	$O(n)$
Get with default	`d.get('x', default)`	$O(1)$	$O(n)$
Get all keys	`d.keys()`	$O(1)$ to get view, $O(n)$ to iterate	—
Get all values	`d.values()`	$O(1)$ to get view, $O(n)$ to iterate	—
Get all items	`d.items()`	$O(1)$ to get view, $O(n)$ to iterate	—
Length	`len(d)`	$O(1)$	—
Pop last added item	`d.popitem()`	$O(1)$	—
Merge	`d.update(other)`	$O(\text{len}(other))$	—
Copy	`d.copy()`	$O(n)$	—
Clear	`d.clear()`	$O(n)$	—
setdefault	`d.setdefault('x', [])`	$O(1)$	$O(n)$

The $O(n)$ worst case only happens with pathological hash collisions; in practice treat these operations as $O(1)$.

Frequency counting

```
# vanilla dictionary
freq = {}
for x in arr:
    freq[x] = freq.get(x,0) + 1

# with defaultdict
from collections import defaultdict
freq = defaultdict(int) # int as factory means missing keys default to int
  ↪ () == 0
for x in arr:
    freq[x] += 1

# counter (especially for frequency calc)
from collections import Counter
freq = Counter(arr)
```

```
from collections import Counter

c = Counter("aabbbc")
print(c) # Counter({'b': 3, 'a': 2, 'c': 1})
print(c.most_common(2)) # [('b', 3), ('a', 2)]
```

1.2 Sets

```
s = {1, 2, 3}   # initialisation with values
s = set()       # empty set; note that {} creates an empty dict, not a set
```

Common Operations

Operation	Syntax	Average case	Worst case
Add element	`s.add(4)`	$O(1)$	$O(n)$
Remove element	`s.remove(4)` (raises `KeyError` if missing)	$O(1)$	$O(n)$
Discard element	`s.discard(4)` (no error if missing)	$O(1)$	$O(n)$
Check membership	`4 in s`	$O(1)$	$O(n)$
Length	`len(s)`	$O(1)$	—
Pop an arbitrary element	`s.pop()`	$O(1)$	—
Union	`s1 | s2`	$O(\text{len}(s1) + \text{len}(s2))$	—
Intersection	`s1 & s2`	$O(\min(\text{len}(s1), \text{len}(s2)))$	—
Difference	`s1 - s2`	$O(\text{len}(s1))$	—
Symmetric difference	`s1 ^ s2`	$O(\text{len}(s1) + \text{len}(s2))$	—
Subset check	`s1 <= s2`	$O(\text{len}(s1))$	—
Copy	`s.copy()`	$O(n)$	—

Dictionary keys and set elements must be *hashable*: numbers, strings and tuples (of hashables) are; lists, sets and dicts are not.

2 Arrays

2.1 Static Arrays

- Fixed size, decided at the time of creation, allocated as a contiguous block of memory.
- Access by index is $O(1)$.
- Insertion/deletion in the middle is $O(n)$, as a lot of values need to be shifted.
- `numpy.ndarrays` behave like static arrays: resizing means allocating a new array and copying, not growing the old one.
- Python `lists` are dynamic arrays (next section).

2.2 Dynamic Arrays

- Same contiguous memory idea, but it can resize.
- If we exceed the capacity, it allocates a bigger block and copies everything over. The textbook choice is to double the size; CPython actually over-allocates by only about 12.5%, which is still enough for amortized $O(1)$ appends.
- Append is $O(1)$ *amortized*, not worst-case: the single append that triggers a resize costs $O(n)$, but this averages out over many appends. (A static array has no append at all.)
- Inserting at the front is $O(n)$ in both cases, as every element has to be shifted.

2.3 Common Operations on Python Lists

Category	Operation	Time Complexity	Notes
Access	`arr[i]`	$O(1)$	Direct index access
	`len(arr)`	$O(1)$	Python lists track length, not counted each time
	`arr[a:b]`	$O(k)$	Slicing creates a new list, doesn't view the original
Adding Elements	`arr.append(x)`	$O(1)$ amortized	Adds at the end
	`arr.extend(iterable)`	$O(k)$ amortized	Adds each element of another iterable
	`arr.insert(i, x)`	$O(n)$	Everything from the index onwards needs to be shifted
Removing Elements	`arr.pop()`	$O(1)$	Removes from the end, no shift needed
	`arr.pop(i)`	$O(n)$	Everything past the index needs to be shifted
	`arr.remove(x)`	$O(n)$	Searches for the first match, then shifts
	`del arr[i]`	$O(n)$	Same cost as pop
	`arr.clear()`	$O(n)$	—

Category	Operation	Time Complexity	Notes
Search	`x in arr`	$O(n)$	Linear search
	`arr.index(x)`	$O(n)$	Returns position
	`arr.count(x)`	$O(n)$	—
Reordering	`arr.sort()`	$O(n \log n)$	TimSort under the hood, stable
	`sorted(arr)`	$O(n \log n)$	Returns a new list, instead of in-place sorting
	`arr.reverse()`	$O(n)$	—
	`arr[::-1]`	$O(n)$	Same idea, but returns a new list
Combining	`arr1 + arr2`	$O(n+m)$	Creates a new list
	`arr * k`	$O(nk)$	—

Note: `pop(0)` and `insert(0, x)` are $O(n)$, not $O(1)$, since the whole list has to shift.

2.4 Implementation of a Dynamic Array

```
class DynamicArray:

    def __init__(self, capacity: int):
        self.capacity = max(1, capacity)   # a capacity of 0 could never
 ↪ grow by doubling
        self.length = 0
        self.arr = [0]*self.capacity

    def get(self, i: int) -> int:
        if not 0 <= i <= self.length-1:
            raise IndexError('Index out of range')
        return self.arr[i]

    def set(self, i: int, n: int) -> None:
        if not 0 <= i < self.length:
            raise IndexError('Index out of range')
        self.arr[i] = n

    def pushback(self, n: int) -> None:
        if self.length == self.capacity:
            self.resize()

        self.arr[self.length] = n
        self.length += 1

    def popback(self) -> int:
        if self.length == 0:
            raise IndexError('pop from empty array')
        self.length -= 1
        return self.arr[self.length]

    def resize(self) -> None:
        self.capacity = 2*self.capacity
        new_arr = [0]* self.capacity

        for i in range(self.length):
            new_arr[i] = self.arr[i]
        self.arr = new_arr

    def getSize(self) -> int:
        return self.length

    def getCapacity(self) -> int:
        return self.capacity
```

3 Strings

3.1 Basics of Strings

- Strings are immutable, so every modification on them creates a new string.
- So, `s += char` in a loop is $O(n^2)$ in the worst case. (CPython sometimes optimises this in place, but that is an implementation detail you should not rely on.)

```
s = "hello"
s[0] = 'H' # TypeError: 'str' object does not support item assignment
```

Fix for building strings in a loop

```
# bad method - O(n^2)
result = ""
for c in some_list:
    result += c

# good method - O(n)
result = []
for c in some_list:
    result.append(c)
result = "".join(result)
```

Access and Slicing

```
s = "hello world"

s[0]        # 'h', O(1)
s[-1]       # 'd', O(1)
s[2:5]      # 'llo', O(k), slicing creates a new string
s[::-1]     # reverses the string, O(n)
s[::2]      # 'hlowrd', every second character
len(s)      # length, O(1)
```

3.2 Common Built-in Methods

Case and Formatting

```
s = "hello world"
s.upper()         # "HELLO WORLD"
s.lower()         # "hello world"
s.title()         # "Hello World"
s.capitalize()    # "Hello world"
s.swapcase()      # "HELLO WORLD", swaps upper/lower
```

Whitespace and cleaning

```
s.strip()           # removes leading and trailing whitespaces
s.strip('xyz')      # removes leading and trailing chars from the given
 ↪ set
s.lstrip()          # removes leading whitespaces
s.rstrip()          # removes trailing whitespaces
```

Searching

```
s = "hello world"
s.find('lo')         # 3, first occurrence, returns -1 if not found
s.rfind('lo')        # 3, last occurrence, returns -1 if not found
s.index('lo')        # same as find, but raises ValueError if not found
'lo' in s            # O(n), membership check
s.count('l')         # 3, counts non-overlapping occurrences, O(n)
s.startswith('hel')  # O(k)
s.endswith('rld')    # O(k)
```

Splitting and Joining

```
s = 'hello world'

s.split()                  # ['hello', 'world'], splits on white space
s.split(',')               # splits on a specified delimiter
s.split(',', 1)            # maxsplit, only splits once
s.rsplit(',', 1)           # splits from the right
s.splitlines()             # splits on line breaks
','.join(['a', 'b', 'c'])  # 'a,b,c', O(n)
```

`s.split()` also handles multiple spaces, leading/trailing whitespaces pretty cleanly.

Replacing

```
s.replace('l', 'L')       # replaces all occurrences, O(n)
s.replace('l', 'L', 1)    # replace only the first occurrence
```

Checking character types

```
c.isalpha()         # is alphabet
c.isdigit()         # is digit
c.isalnum()         # is digit or alphabet
c.isupper()         # is uppercase
c.islower()         # is lowercase
c.isspace()         # is whitespace
```

These return `True` only if *every* character satisfies the condition (and the string is non-empty).

Padding and Alignment

```
s.zfill(5)          # '00042' for '42', pads with zeros on the left
s.ljust(10, '*')    # left-justify, pad right
s.rjust(10, '*')    # right-justify, pad left
s.center(10, '*')   # center with padding
```

3.3 Character ↔ ASCII Conversion

```
ord('a')   # 97, character to ASCII/Unicode code point
chr(97)    # 'a', code point to character

# common pattern: map 'a'-'z' to indices 0-25
ch = 'c'
index = ord(ch) - ord('a')
letter = chr(index + ord('a'))
```

3.4 String ↔ list conversion (in-place-style manipulation)

```
s = 'hello'
chars = list(s)     # ['h', 'e', 'l', 'l', 'o']
chars[0] = 'H'
s = ''.join(chars)  # 'Hello'
```

3.5 String Comparison

```
"abc" == "abc"          # True, value comparison
"abc" < "abd"           # True, lexicographic comparison, character by
  ↪ character

sorted("cab")           # ['a', 'b', 'c'], returns a list, not a string
''.join(sorted("cab"))  # 'abc'
```

4 Sorting

4.1 Time Complexities

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	Yes
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + k)$	Yes
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n)$	Yes (depends on inner sort)
Tim Sort (Python's `sort()`)	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes

Stability: a sort is stable if elements with equal keys keep their original relative order (this matters when sorting records, e.g. by age after sorting by name).

Notes on the table: the quick sort space is the average recursion depth (worst case $O(n)$); bubble and insertion sort reach $O(n)$ only on (nearly) sorted input; counting sort is stable only in its prefix-sum form (shown under radix sort); k is the value range for counting/bucket sort and the number of digits for radix sort.

4.2 Comparison based Algorithms

Bubble Sort

- Repeatedly swaps adjacent elements if they're out of order.
- Each pass "bubbles" the largest unsorted element to its correct position at the end.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break  # already sorted, stop early
    return arr
```

Selection Sort

- Repeatedly finds the minimum of the unsorted portion and swaps it into place.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1,n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Insertion Sort

- Builds the sorted array, one element at a time.
- Inserts each new element into its correct position among the already-sorted part.

```
def insertion_sort(arr):
    for i in range(1,len(arr)):
        key = arr[i]
        j = i-1
        while j>= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
```

Merge Sort

- DnC: Divide the array into halves, sort them and merge back.
- Stable (because of the `<=` in the merge step), but needs $O(n)$ extra space.

```
def merge_sort(arr):
```

```
    if len(arr) <= 1:
        return arr

    mid = len(arr)//2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left,right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result
```

Quick Sort

- DnC: Pick a pivot, partition the array such that smaller elements are to the left of the pivot and the rest are on the right.
- Recurse on each step.
- With the last element as pivot, already sorted input is the worst case ($O(n^2)$ time and $O(n)$ recursion depth); picking a random pivot avoids this in practice.

```
def quick_sort(arr, low=0, high=None):
    if high is None:
        high = len(arr)-1
    if low < high:
        pivot_idx = partition(arr,low,high)
        quick_sort(arr,low,pivot_idx-1)
        quick_sort(arr,pivot_idx+1,high)
    return arr

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[high] = arr[high], arr[i+1]
    return i+1
```

Heap Sort

- Builds a max-heap from the array, then repeatedly extract the max and place it at the end.

```
def heap_sort(arr):
    n = len(arr)
    for i in range(n//2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n-1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)
    return arr

def heapify(arr, n, i):
    largest = i
    left, right = 2*i + 1, 2*i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
```

4.3 Non-comparison based Algorithms

Counting Sort

- Counts the occurrences of each value and then reconstructs the sorted array. Works for non-negative integers; k is the maximum value.

```
def counting_sort(arr):
    if not arr: return arr

    max_val = max(arr)
    count = [0]*(max_val + 1)

    for x in arr:
        count[x] += 1

    result = []
    for val, c in enumerate(count):
        result.extend([val]*c)
    return result
```

Radix Sort

- Sorts integers digit by digit, least significant digit first, using a **stable** counting sort on each digit. $O(d \cdot (n + b))$ for d digits in base b.
- The prefix-sum form of counting sort below is what makes counting sort stable: `count[d]` becomes the end position of digit `d`, and the output is filled from the back.

```
def radix_sort(arr):
    """Non-negative integers only."""
    if not arr: return arr
    max_val = max(arr)
    exp = 1
    while max_val // exp > 0:
        arr = counting_sort_by_digit(arr, exp)
        exp *= 10
    return arr

def counting_sort_by_digit(arr, exp):
    count = [0]*10
    for x in arr:
        count[(x // exp) % 10] += 1
    for d in range(1, 10):
        count[d] += count[d-1]           # prefix sums
    out = [0]*len(arr)
    for x in reversed(arr):              # go backwards to stay stable
        d = (x // exp) % 10
        count[d] -= 1
        out[count[d]] = x
    return out
```

Bucket Sort

- Spread the values into buckets by range, sort every bucket, then concatenate. $O(n+k)$ on average for roughly uniformly distributed input, $O(n^2)$ if everything lands in one bucket.

```
def bucket_sort(arr):
    """Floats in [0, 1)."""
    n = len(arr)
    if n == 0: return arr
    buckets = [[] for _ in range(n)]
    for x in arr:
        buckets[int(x * n)].append(x)
    result = []
    for b in buckets:
        result.extend(sorted(b))
    return result
```

5 Linked Lists

5.1 Basic Structure

- Nodes hold a value and a pointer to the next node; there is no contiguous memory and no random access.
- `get(i)`, `insertTail` and `remove(i)` are $O(n)$; `insertHead` is $O(1)$ (keeping a tail pointer would make `insertTail` $O(1)$ too).

```
# linked lists
from typing import List

class Node:
    def __init__(self, val=None):
        self.val = val
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def get(self, index: int) -> int:
        if index < 0:
            return -1
        temp = self.head
        i = 0
        while i < index and temp:
            temp = temp.next
            i += 1
        return temp.val if temp else -1

    def insertHead(self, val: int) -> None:
        temp = self.head
        self.head = Node(val)
        self.head.next = temp

    def insertTail(self, val: int) -> None:
        if not self.head:
            self.head = Node(val)
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = Node(val)

    def remove(self, index: int) -> bool:
        if not self.head or index < 0:
            return False
        if index == 0:
            self.head = self.head.next
            return True
        curr = self.head
        i = 0
        while curr.next and i < index - 1:
            curr = curr.next
            i += 1
        if curr.next is None:
            return False
        curr.next = curr.next.next
        return True

    def getValues(self) -> List[int]:
        result = []
        curr = self.head
        while curr:
            result.append(curr.val)
            curr = curr.next
        return result
```

5.2 Reversing a Linked List

```
def reverseLL(head: Node) -> Node:
    prev, current = None, head

    while current:
        nextNode = current.next
        current.next = prev
        prev = current
        current = nextNode

    return prev
```

5.3 Detecting Cycle

- Floyd's tortoise and hare: `slow` moves one step, `fast` moves two; they meet if and only if the list has a cycle. $O(n)$ time, $O(1)$ space.

```
def detectCycle(head: Node) -> bool:
    slow = fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

        if slow == fast: return True

    return False
```

6 Queue

6.1 Introduction

- FIFO (First In, First Out) functionality
- Heavily used in Graph BFS Traversals. `list.pop(0)` is $O(n)$, so a plain list is a poor fit.
- We mostly use the `deque` (pronounced as deck) functionality from `collections` library.

6.2 Some important features/functions on `deque`

```
from collections import deque
d = deque([1,2,3])
```

Operation	Time Complexity	Notes
`d.append(4)`	$O(1)$	Appends to the right
`d.appendleft(0)`	$O(1)$	Appends to the left
`d.pop()`	$O(1)$	Removes and returns the rightmost element
`d.popleft()`	$O(1)$	Removes and returns the leftmost element
`d.extend(iterable)`	$O(k)$	Appends multiple elements to the right
`d.extendleft(iterable)`	$O(k)$	Appends multiple elements to the left (in reverse order)
`d.insert(i, x)`	$O(n)$	Inserts x at index i
`d.remove(x)`	$O(n)$	Removes the first occurrence of x

Operation	Time Complexity	Notes
`d.rotate(n)`	$O(k)$	Rotates by n steps (negative n means rotate left)
`d.reverse()`	$O(n)$	Reverses the deque in place
`d.clear()`	$O(n)$	Removes all elements
`d.count(x)`	$O(n)$	Counts all occurrences of x
`d.index(x)`	$O(n)$	Finds the index of x
`d.copy()`	$O(n)$	Shallow copy of the deque
`d[i]`	$O(n)$	Access by index, deque is not optimized for random access
`len(d)`	$O(1)$	Length of the deque

6.3 An important note

- `deque(maxlen=N)` creates a bounded `deque`, which is heavily used in sliding window approaches. If we append beyond `maxlen`, elements from the opposite end are discarded.

```
from collections import deque

d = deque(maxlen=3)
d.append(1); d.append(2); d.append(3); d.append(4)

print(d) # deque([2, 3, 4], maxlen=3), the 1 got dropped off
```

6.4 Implementation of Queues using LinkedLists

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
        self.size = 0

    def enqueue(self, val):
        newNode = Node(val)
        if not self.rear:
            self.front = self.rear = newNode
        else:
            self.rear.next = newNode
            self.rear = newNode
        self.size += 1

    def dequeue(self) -> int:
        if not self.front:
            raise IndexError("dequeue from an empty queue")
        val = self.front.val
        self.front = self.front.next
        if not self.front:
            self.rear = None
        self.size -= 1
        return val

    def peek(self) -> int:
        if not self.front:
            raise IndexError("peek from an empty queue")
        return self.front.val

    def is_empty(self) -> bool:
        return self.front is None
```

6.5 Implementation of Queues using two stacks

- `dequeue` and `peek` are $O(1)$ amortized: every element is moved from `in_stack` to `out_stack` at most once.

```
class QueueUsingStacks:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def enqueue(self, val) -> None:
        self.in_stack.append(val)

    def dequeue(self) -> int:
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        if not self.out_stack:
            raise IndexError("dequeue from an empty queue")
        return self.out_stack.pop()

    def peek(self) -> int:
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        if not self.out_stack:
            raise IndexError("peek from an empty queue")
        return self.out_stack[-1]

    def is_empty(self) -> bool:
        return not self.in_stack and not self.out_stack
```

6.6 Implementation of Circular Queue

- A fixed-capacity ring buffer: `front` plus `count` determine the rear index, which avoids the empty-vs-full ambiguity of a two-pointer design.

```
class CircularQueue:
    def __init__(self, k):
        self.q = [None]*k
        self.capacity = k
        self.front = 0
        self.count = 0

    def enqueue(self,val) -> None:
        if self.count == self.capacity:
            raise OverflowError("queue is full")
        rear = (self.front + self.count) % self.capacity
        self.q[rear] = val
        self.count += 1

    def dequeue(self) -> int:
        if self.count == 0:
            raise IndexError("Empty Queue")
        val = self.q[self.front]
        self.front = (self.front + 1) % self.capacity
        self.count -= 1
        return val

    def is_empty(self) -> bool:
        return self.count == 0

    def is_full(self) -> bool:
        return self.count == self.capacity
```

7 Double-ended Queues

7.1 Introduction

- Double ended queues give us $O(1)$ functionality to append/pop on both left and right ends.
- Python's `collections.deque` is a double ended queue.

7.2 Implementation using Doubly Linked Lists

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None
        self.prev = None

class Deque:
    def __init__(self):
        self.front = None
        self.rear = None
        self.size = 0

    def append(self,val: int) -> None:
        newNode = Node(val)
        if not self.rear:
            self.front = self.rear = newNode
        else:
            newNode.prev = self.rear
            self.rear.next = newNode
            self.rear = newNode
        self.size += 1

    def appendleft(self, val: int) -> None:
        newNode = Node(val)
        if not self.rear:
            self.front = self.rear = newNode
        else:
            newNode.next = self.front
            self.front.prev = newNode
            self.front = newNode
        self.size += 1

    def pop(self) -> int:
        if not self.rear:
            raise IndexError("empty queue")
        val = self.rear.val
        self.rear = self.rear.prev
        if not self.rear:
            self.front = None
        else:
            self.rear.next = None
        self.size -= 1
        return val

    def popleft(self) -> int:
        if not self.rear:
            raise IndexError("empty queue")
        val = self.front.val
        self.front = self.front.next
        if not self.front:
            self.rear = None
        else:
            self.front.prev = None
        self.size -= 1
        return val

    def is_empty(self) -> bool:
        return self.size == 0
```

```
# all operations here are in O(1)
```

8 Stack

8.1 Stack Basics

- Stacks follow the Last In, First Out (LIFO) protocol.
- We can generally use Python's default Lists as stacks.
- We may also use `collections.deque`.
- `append` and `pop` at the end of a list are $O(1)$ (amortized for `append`), so a plain list is enough for most problems.

```
stack = []
stack.append(1)
stack.append(2)
stack.append(3)

print(stack.pop()) # pop and return the top element
print(stack[-1]) # peek
print(len(stack)) #size
print(not stack) #is_empty check
```

```
# stacks using collections.deque
from collections import deque

stack = deque()
stack.append(1)
stack.append(2)
stack.append(3)

print(stack.pop()) # 3
print(stack[-1]) #peek
```

8.2 Implementation using LinkedLists

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

class Stack:
    def __init__(self):
        self.top = None
        self.size = 0

    def push(self, val: int) -> None:
        newNode = Node(val)
        newNode.next = self.top
        self.top = newNode
        self.size += 1

    def pop(self) -> int:
        if not self.top:
            raise IndexError("Empty Stack")
        val = self.top.val
        self.top = self.top.next
        self.size -= 1
        return val

    def peek(self) -> int:
        if not self.top:
            raise IndexError("Empty Stack")
        return self.top.val

    def is_empty(self) -> bool:
        return self.top is None
```

9 Heaps / Priority Queues

9.1 Basics

- A binary heap is a complete binary tree in which every parent is $\leq$ its children (min-heap), so the minimum always sits at the root.
- Unlike LIFO or FIFO, PQs don't care about the insertion order but rather the priority order.

9.2 `heapq` module

- `heapq` works directly on python `list`, turning it into a binary heap in-place.

```
import heapq

heap = []

heapq.heappush(heap, 5)
heapq.heappush(heap, 1)
heapq.heappush(heap, 3)

print(heapq.heappop(heap)) # 1 -> smallest
print(heapq.heappop(heap)) # 3
```

heapq functions

Function	Time Complexity	Notes
`heapq.heappush(heap, x)`	$O(\log n)$	Push x onto heap
`heapq.heappop(heap)`	$O(\log n)$	Pop and return smallest item
`heapq.heappushpop(heap, x)`	$O(\log n)$	Push x and then pop the smallest (more efficient than push + pop)
`heapq.heapreplace(heap, x)`	$O(\log n)$	Pop the smallest, and then push x
`heapq.heapify(list)`	$O(n)$	Convert an existing list into a heap, in-place
`heapq.nlargest(k, iterable)`	$O(n \log k)$	Returns the k largest elements
`heapq.nsmallest(k, iterable)`	$O(n \log k)$	Returns the smallest k elements
`heap[0]`	$O(1)$	Peek (smallest element sits at index 0, not -1)

max-heap trick

- `heapq` only gives us min-heap, for max-heap, we can simply negate the values on the way in and way out.

```
import heapq

heap = []
heapq.heappush(heap, -5)
heapq.heappush(heap, -1)
heapq.heappush(heap, -3)

print(-heapq.heappop(heap)) # 5 -> largest
print(-heapq.heappop(heap)) # 3 -> second largest
```

Priority Queues with custom order (tuple)

- tuples should be of the format `(priority, value)`

```
import heapq

heap = []

heapq.heappush(heap, (2, "task x"))
heapq.heappush(heap, (1, "task l"))
heapq.heappush(heap, (3, "task c"))

print(heapq.heappop(heap)) # (1, 'task l')
```

- If two priorities tie, Python compares the next tuple element, which raises `TypeError` for non-comparable items (e.g. custom objects). Use `(priority, counter, item)` with an increasing counter as a tie-breaker.

9.3 Implementation of heaps from scratch

- A binary heap is stored as an array, where the children of index i element are at $2i+1$, and $2i+2$, and its parent lives at $(i-1)//2$.
- `push` adds a value at the end and bubbles up (sift up).
- `pop` swaps the last value with the root and bubbles down (sift down).
- Both are $O(\log n)$, as the height of the tree is $\log n$.

```
class MinHeap:
    def __init__(self):
        self.heap = []

    def _parent(self, i):
```

```
        return (i-1)//2

    def _left(self, i):
        return 2*i+1

    def _right(self, i):
        return 2*i+2

    def push(self, val):
        self.heap.append(val)
        self._sift_up(len(self.heap)-1)

    def _sift_up(self, i):
        while i> 0 and self.heap[i] < self.heap[self._parent(i)]:
            p = self._parent(i)
            self.heap[i], self.heap[p] = self.heap[p], self.heap[i]
            i = p

    def pop(self):
        if not self.heap:
            raise IndexError("Empty Heap")
        top = self.heap[0]
        last = self.heap.pop()
        if self.heap:
            self.heap[0] = last
            self._sift_down(0)
        return top

    def _sift_down(self, i):
        n = len(self.heap)
        while True:
            smallest = i
            l, r = self._left(i), self._right(i)
            if l < n and self.heap[l] < self.heap[smallest]:
                smallest = l
            if r < n and self.heap[r] < self.heap[smallest]:
                smallest = r
            if smallest == i:
                break
            self.heap[i], self.heap[smallest] = self.heap[smallest], self.
↪ heap[i]
            i = smallest

    def peek(self):
        if not self.heap:
            raise IndexError("Empty Heap")
        return self.heap[0]

    def is_empty(self):
        return len(self.heap) == 0
```

10 Binary Trees

10.1 Binary Trees Basics

```
# class for TreeNode
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

# adding stuff
tree = TreeNode(1)
tree.left = TreeNode(2)
tree.right = TreeNode(3)
tree.left.left = TreeNode(4)
```

10.2 Binary Trees Traversal

```
# Preorder Traversal - DFS
def preorder_dfs(root: TreeNode) -> None:
    if root:
        print(root.val, end=" ")
        preorder_dfs(root.left)
        preorder_dfs(root.right)
```

```
# Inorder Traversal - DFS
def inorder_dfs(root: TreeNode) -> None:
    if root:
        inorder_dfs(root.left)
        print(root.val, end=" ")
        inorder_dfs(root.right)
```

Note - Inorder traversal of a BST gives a sorted array.

```
# Postorder Traversal - DFS
def postorder_dfs(root: TreeNode) -> None:
    if root:
        postorder_dfs(root.left)
        postorder_dfs(root.right)
        print(root.val, end=" ")
```

```
# BFS Traversal using Queue
from collections import deque
def bfs(root: TreeNode) -> None:
    if not root: return

    queue = deque([root])
    while queue:
        node = queue.popleft()
        print(node.val, end=" ")
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
```

```
# iterative inorder traversal using explicit stack
from typing import List

def iterative_inorder(root: TreeNode) -> List[int]:
    result, stack = [], []
    curr = root
    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        result.append(curr.val)
        curr = curr.right
    return result
```

10.3 Constructing a Binary Tree from preorder and inorder traversals

- Preorder gives the root first; its position in the inorder sequence splits the remaining nodes into the left and right subtrees. Assumes all values are unique.
- The naive version is $O(n^2)$ (`index` plus slicing); the hashmap version below is $O(n)$.

```
# naive implementation
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def build_tree(preorder, inorder):
    if not preorder or not inorder:
        return None

    root_val = preorder[0]
    root = TreeNode(root_val)

    mid = inorder.index(root_val)

    root.left = build_tree(preorder[1:mid+1], inorder[:mid])
    root.right = build_tree(preorder[mid+1:], inorder[mid+1:])

    return root
```

```
# optimal implementation
# we use a hashmap to lookup inorder index and used only pointers as
  ↪ opposed to copying the whole arrays

def build_tree(preorder, inorder):
    inorder_index = {val: i for i, val in enumerate(inorder)}
    pre_idx = 0

    def helper(left, right):
        nonlocal pre_idx
        if left > right:
            return None

        root_val = preorder[pre_idx]
        root = TreeNode(root_val)
        pre_idx += 1

        mid = inorder_index[root_val]

        root.left = helper(left, mid-1)
        root.right = helper(mid+1, right)

        return root

    return helper(0, len(preorder)-1)
```

10.4 Height of a Binary Tree

- Here the height counts **nodes** on the longest root-to-leaf path (an empty tree has height 0). To count edges instead, return `-1` for an empty tree.

```
def height(root: TreeNode) -> int:
    if not root: return 0

    leftHt = height(root.left)
    rightHt = height(root.right)

    return 1 + max(leftHt, rightHt)
```

10.5 Number of Nodes in a Binary Tree

```
def numNodes(root: TreeNode) -> int:
    if not root: return 0

    leftNums = numNodes(root.left)
    rightNums = numNodes(root.right)

    return 1 + leftNums + rightNums
```

10.6 Is Identical

```
def isIdentical(p: TreeNode, q: TreeNode) -> bool:
    if not p or not q: return p == q

    return (
        p.val ==  q.val and
        isIdentical(p.left, q.left) and
        isIdentical(p.right, q.right)
    )
```

10.7 Is Subtree

- Test `isIdentical` at every node of `root`: $O(m \cdot n)$ worst case for trees with n and m nodes.

```
def isSubtree(root: TreeNode, subRoot: TreeNode) -> bool:

    if not subRoot: return True
    if not root: return False

    if (root.val == subRoot.val and isIdentical(root, subRoot)): return
  ↪ True

    return (
        isSubtree(root.left, subRoot) or
        isSubtree(root.right, subRoot)
    )
```

10.8 Diameter of a binary tree

- Maximum distance (in edges) between any two nodes; the path may or may not pass through the root. The longest path through a node is `left_height + right_height`.

```
def diameter(root: TreeNode) -> int:
    result = 0
    def depth(root: TreeNode) -> int:
        nonlocal result
        if not root:
            return 0
        left = depth(root.left)
        right = depth(root.right)
        result = max(result, left+right)
        return 1 + max(left, right)
    depth(root)
    return result
```

10.9 Check if balanced

- Balanced tree: for every node, the height difference between left and right subtrees is at most 1.
- Naive approach recomputes height at every node $\rightarrow O(n^2)$. Better: compute height and check balance in a single bottom-up pass $\rightarrow O(n)$.

```
def is_balanced(root: TreeNode) -> bool:
    def height(node: TreeNode) -> int:
        if not node:
            return 0
        left = height(node.left)
        if left == -1:
            return -1
        right = height(node.right)
        if right == -1:
            return -1
        if abs(left - right) > 1:
            return -1
        return 1 + max(left, right)

    return height(root) != -1
```

11 Binary Search Trees

11.1 Basics

- For every node, all values in the left subtree are smaller and all values in the right subtree are larger (`left < node < right`). We assume distinct keys; the `insert` below ignores duplicates.
- Search, insert and delete are $O(h)$ for height h: $O(\log n)$ when balanced, $O(n)$ for a degenerate tree (e.g. keys inserted in sorted order).

11.2 Implementation from Scratch

```
class TreeNode:
    def __init__(self, val: int):
        self.val = val
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, val: int) -> None:
        self.root = self._insert(self.root, val)

    def _insert(self, node: TreeNode, val: int) -> TreeNode:
        if node is None:
            return TreeNode(val)
        if val < node.val:
            node.left = self._insert(node.left, val)
        elif val > node.val:
            node.right = self._insert(node.right, val)
        # equal: duplicate, ignored
        return node

    def search(self, val: int) -> TreeNode:
        return self._search(self.root, val)

    def _search(self, node: TreeNode, val: int) -> TreeNode:
        if node is None or node.val == val:
            return node
        if val < node.val:
            return self._search(node.left, val)
        return self._search(node.right, val)

    def delete(self, val: int) -> None:
        self.root = self._delete(self.root, val)

    def _delete(self, node: TreeNode, val: int) -> TreeNode:
        if node is None: return None
        if val < node.val:
            node.left = self._delete(node.left, val)
        elif val > node.val:
            node.right = self._delete(node.right, val)
        else: # found the match, now three cases
            if not node.left:
                return node.right
            if not node.right:
                return node.left
            #when both children exist
            # find inorder successor (smallest in the right subtree)
            successor = node.right
            while successor.left:
                successor = successor.left
            node.val = successor.val
            node.right = self._delete(node.right, successor.val)
        return node
```

11.3 Validate a BST

```
# check if a given tree is actually BST
def is_valid_bst(node: TreeNode, low = float('-inf'), high = float('inf'))
  ↪  -> bool:
    if not node: return True

    if not (low < node.val < high): return False

    return is_valid_bst(node.left, low, node.val) and is_valid_bst(node.
  ↪ right, node.val, high)
```

11.4 Lowest Common Ancestor (LCA)

- The first version works for any binary tree (it assumes both `p` and `q` are present): $O(n)$.

```
def lca(root:TreeNode, p:int, q:int) -> TreeNode:
    if root is None or root.val == p or root.val == q:
        return root

    left = lca(root.left, p, q)
    right = lca(root.right, p, q)

    if left and right: return root

    return left if left else right
```

For a BST we can use the ordering instead of searching both subtrees, which is $O(h)$:

```
def lca_bst(root: TreeNode, p: int, q: int) -> TreeNode:
    while root:
        if p < root.val and q < root.val:
            root = root.left
        elif p > root.val and q > root.val:
            root = root.right
        else:
            return root   # p and q are split by root (or one equals root)
    return None
```

12 Graphs

12.1 Basics

- A graph is a set of nodes (vertices) connected via edges. Below, V is the number of vertices and E the number of edges.
- No hierarchy, unlike trees.
- Types of Graphs:
  - Directed
  - Undirected
  - Weighted
  - Unweighted
  - Cyclic
  - Acyclic
  - Connected
  - Sparse vs Dense

12.2 Representation

Adjacency List

- A dictionary mapping each node to a list of its neighbors. Efficient for sparse graphs.

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
}
```

- For weighted graphs, we store `(neighbor, weight)` pairs.

```
graph = {
    'A': [('B', 4), ('C', 1)],
    'B': [('A', 4), ('D', 2)],
    'C': [('A', 1), ('D', 5)],
    'D': [('B', 2), ('C', 5)]
}
```

Adjacency Matrix

- A 2d array, where `matrix[i][j] = 1` if there is an edge between `i` and `j`.

```
# 4 nodes: 0, 1, 2, 3
matrix = [
    [0, 1, 1, 0],
    [1, 0, 0, 1],
    [1, 0, 0, 1],
    [0, 1, 1, 0]
]
```

Edge List

- Just a flat list of edges. (rarely used for traversal but common way of input in Qs)

```
edges = [('A', 'B'), ('A', 'C'), ('B', 'D'), ('C', 'D')]
```

Adjacency list vs. adjacency matrix

	Adjacency list	**Adjacency matrix**
Space	$O(V+E)$	$O(V^2)$
Is there an edge u–v?	$O(\deg(u))$	$O(1)$
Iterate over neighbours of u	$O(\deg(u))$	$O(V)$
Best for	sparse graphs	dense graphs

Building an adjacency list from edge list

```
from collections import defaultdict
from typing import List

def build_adj_list(edges: List, directed: bool = False) -> dict:
    graph = defaultdict(list)
    for u, v in edges:
        graph[u].append(v)
        if directed:
            graph[v]                # touch v, so that sink nodes also
  ↪ become keys
        else:
            graph[v].append(u)
    return graph

# the same with a plain dict (no import needed)
def build_adj_list_plain(edges: List, directed: bool = False) -> dict:
    graph = {}
    for u, v in edges:
        graph.setdefault(u, []).append(v)
        if directed:
            graph.setdefault(v, [])
        else:
            graph.setdefault(v, []).append(u)
    return graph
```

12.3 Traversals

- DFS and BFS both run in $O(V+E)$ time on an adjacency list; `visited` prevents revisiting nodes (and looping forever on cycles).
- Recursive DFS is limited by Python's recursion limit (about 1000 frames by default); use the iterative version for deep graphs.
- The cycle detection, topological sort and traversal code below looks up `graph[node]` for every node, so every node must be a key of `graph` (this is why the builder above also adds sink nodes).

DFS (Recursive)

```
def dfs(graph, node, visited=None, result=None):
    if visited is None:
        visited, result = set(), []
    visited.add(node)
    result.append(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited, result)
    return result
```

DFS (Iterative using Stack)

- Visits the neighbours in the opposite order to the recursive version (the last one pushed is popped first).

```
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    result = []
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            result.append(node)
            for neighbor in graph[node]:
                if neighbor not in visited:
                    stack.append(neighbor)
    return result
```

BFS (always iterative, uses Queue)

- BFS visits nodes in order of distance from `start`, so it finds shortest paths in unweighted graphs.

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    q = deque([start])
    result = []
    while q:
        node = q.popleft()
        result.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                q.append(neighbor)
    return result
```

12.4 Common Algorithms

Detect a Cycle (Undirected Graph)

- DFS; a visited neighbour that is not the parent means a cycle. (Parallel edges between the same two nodes are not detected by this version.)

```
def detect_cycle_undirected(graph):
    visited = set()

    def dfs(node, parent):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                if dfs(neighbor, node):
                    return True
            elif neighbor != parent:
                return True
        return False

    for node in graph:
        if node not in visited:
            if dfs(node, None): return True
    return False
```

Detect a Cycle (Directed Graph)

- Three colours: WHITE = unvisited, GRAY = on the current DFS path, BLACK = finished. An edge to a GRAY node closes a cycle.

```
def detect_cycle_directed(graph):
    WHITE, GRAY, BLACK = 0, 1, 2
    color = {node: WHITE for node in graph}

    def dfs(node):
        color[node] = GRAY
        for neighbor in graph[node]:
            if color[neighbor] == GRAY:
                return True
            if color[neighbor] == WHITE and dfs(neighbor):
                return True
        color[node] = BLACK
        return False

    for node in graph:
        if color[node] == WHITE:
            if dfs(node): return True
    return False
```

Topological Sort (DAGs Only)

- An ordering in which, for every edge `u -> v`, `u` comes before `v`. DFS gives it as the reverse of the finishing order; $O(V+E)$. On a graph with a cycle the result is meaningless, so check for cycles first (or use Kahn's algorithm).

```
def topological_sort(graph):
    visited = set()
    stack = []

    def dfs(node):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                dfs(neighbor)
        stack.append(node) # add after all descendants are processed

    for node in graph:
        if node not in visited:
            dfs(node)

    return stack[::-1]
```

Kahn's Algorithm (BFS alternative, also detects cycles)

- Repeatedly remove nodes with in-degree 0. If some nodes are never removed, they lie on or behind a cycle.

```
from collections import deque

def topological_sort_kahn(graph, nodes):
    in_degree = {n: 0 for n in nodes}
    for node in graph:
        for neighbor in graph[node]:
            in_degree[neighbor] += 1

    q = deque([n for n in nodes if in_degree[n] == 0])
    result = []

    while q:
        node = q.popleft()
        result.append(node)
        for neighbor in graph.get(node, []):
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                q.append(neighbor)

    if len(result) != len(nodes):
        return None   # cycle detected, no valid topological order
    return result
```

Number of connected Components

- For undirected graphs: every DFS started from an unvisited node explores exactly one component.

```
def count_components(graph, nodes):
    visited = set()
    count = 0

    def dfs(node):
        visited.add(node)
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                dfs(neighbor)

    for node in nodes:
        if node not in visited:
            dfs(node)
            count += 1
    return count
```

12.5 Shortest Paths

Dijkstra's Algorithm (only for non-negative weights)

- Greedy: always settle the closest unsettled node, using a min-heap. $O((V+E)\log V)$.
- Heap entries that became outdated are skipped when popped (`d > dist[node]`) instead of being removed. Unreachable nodes are simply absent from `dist`.

```
import heapq

def dijkstra(graph, start):
    dist = {start: 0}
    heap = [(0, start)]
```

```
    while heap:
        d, node = heapq.heappop(heap)
        if d > dist.get(node, float('inf')):
            continue                    # stale entry, a shorter path was
↪ already found
        for neighbor, weight in graph.get(node, []):
            new_dist = d + weight
            if new_dist < dist.get(neighbor, float('inf')):
                dist[neighbor] = new_dist
                heapq.heappush(heap, (new_dist, neighbor))
    return dist
```

Bellman-Ford Algorithm

- Single-source shortest paths that **allows negative edge weights**. Relax every edge $V-1$ times; if some edge can still be relaxed afterwards, a negative cycle is reachable from the source. $O(VE)$.

```
def bellman_ford(nodes, edges, start):
    """edges: list of (u, v, w). Returns the dist dict, or None on a
↪ negative cycle."""
    dist = {n: float('inf') for n in nodes}
    dist[start] = 0
    for _ in range(len(dist) - 1):
        updated = False
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                updated = True
        if not updated:
            break                       # converged early
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            return None                 # still improving -> negative cycle
    return dist
```

SPFA - Shortest Path Faster Algorithm

- Queue-based Bellman-Ford: only re-process a node when its distance has just improved. Fast on typical graphs, but still $O(VE)$ in the worst case. Also handles negative edges.
- A node entering the queue V times means a negative cycle.

```
from collections import deque

def spfa(graph, nodes, start):
    """graph[u] = list of (v, w). Returns the dist dict, or None on a
↪ negative cycle."""
    dist = {n: float('inf') for n in nodes}
    dist[start] = 0
    q = deque([start])
    in_queue = {start}
    count = {n: 0 for n in nodes}      # how often a node was enqueued
    while q:
        u = q.popleft()
        in_queue.discard(u)
        for v, w in graph.get(u, []):
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if v not in in_queue:
                    count[v] += 1
                    if count[v] >= len(dist):
                        return None    # negative cycle
                    q.append(v)
                    in_queue.add(v)
    return dist
```

Floyd-Warshall Algorithm

- All-pairs shortest paths: after processing `k`, `dist[i][j]` is the shortest path from `i` to `j` that only uses intermediate nodes `0..k`. The `k` loop must be the outermost one.
- $O(V^3)$ time, $O(V^2)$ space; works with negative edges. A negative cycle exists if some `dist[i][i] < 0` at the end.

```
def floyd_warshall(n, edges):
    """Nodes are 0..n-1, edges are (u, v, w). Returns the n x n distance
↪ matrix."""
    INF = float('inf')
    dist = [[INF] * n for _ in range(n)]
    for i in range(n):
        dist[i][i] = 0
    for u, v, w in edges:
        dist[u][v] = min(dist[u][v], w)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist
```

12.6 Minimum Spanning Tree

- For a connected, weighted, undirected graph: $V-1$ edges that connect all vertices with the smallest possible total weight (no cycles).
- Two classic greedy algorithms: Prim grows one tree from a start node by always taking the cheapest edge leaving it; Kruskal sorts all edges and adds each one that does not close a cycle.

Prim's Algorithm

- Heap of candidate edges. $O(E \log V)$. A good fit when the graph is given as an adjacency list.

```
import heapq

def prim(graph, start):
    """graph[u] = list of (v, w), undirected (each edge stored in both
↪ directions).
    Returns (total_weight, mst_edges) for the component containing start.
↪ """
    visited = {start}
    heap = [(w, start, v) for v, w in graph[start]]
    heapq.heapify(heap)
    total, mst = 0, []
    while heap:
        w, u, v = heapq.heappop(heap)
        if v in visited:
            continue
        visited.add(v)
        total += w
        mst.append((u, v, w))
        for nxt, nw in graph[v]:
            if nxt not in visited:
                heapq.heappush(heap, (nw, v, nxt))
    return total, mst
```

Kruskal's Algorithm

- Sorting dominates: $O(E \log E)$. Cycle checks use **union-find** (disjoint set union), which is nearly $O(1)$ per operation with path compression and union by size. A good fit when the graph is given as an edge list.

```
class DSU:
    def __init__(self, nodes):
        self.parent = {n: n for n in nodes}
        self.size = {n: 1 for n in nodes}

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]   # path halving
            x = self.parent[x]
        return x

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False               # already connected, would form a
↪ cycle
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        return True

def kruskal(nodes, edges):
    """edges: list of (u, v, w). Returns (total_weight, mst_edges); a
↪ forest if disconnected."""
    dsu = DSU(nodes)
    total, mst = 0, []
    for u, v, w in sorted(edges, key=lambda e: e[2]):
        if dsu.union(u, v):
            total += w
            mst.append((u, v, w))
    return total, mst
```

12.7 Minimum Height Trees

- Given a tree (connected, acyclic, undirected) with n nodes, choose the root that minimises the height of the rooted tree. There are at most two such roots: the centre(s) of the tree.
- Idea: repeatedly peel off all leaves, like Kahn's algorithm working from the outside in. The one or two nodes left at the end are the answer. $O(n)$.

```
def find_min_height_trees(n, edges):
    if n <= 2:
        return list(range(n))
    adj = [set() for _ in range(n)]
    for u, v in edges:
        adj[u].add(v)
        adj[v].add(u)
    leaves = [i for i in range(n) if len(adj[i]) == 1]
    remaining = n
    while remaining > 2:
        remaining -= len(leaves)
        new_leaves = []
        for leaf in leaves:
            nb = adj[leaf].pop()
            adj[nb].remove(leaf)
            if len(adj[nb]) == 1:
                new_leaves.append(nb)
        leaves = new_leaves
    return leaves
```

13 Bit Manipulation

13.1 Basics

Basic Operators

```
a = 5           # 0101
b = 3           # 0011

a & b           # 1,  0001, and
a | b           # 7,  0111, or
a ^ b           # 6,  0110, xor

~a              # -6, flips all bits; in Python, ~a is defined as -(a+1)

a << 1          # 10, 1010, multiply by 2
a >> 1          # 2,  0010, divide by 2 (floor)
```

XOR Properties

- XOR is commutative and associative, so the order does not matter and equal pairs cancel out.

```
a ^ a           # 0, xor anything with itself is 0
a ^ 0           # a, xor with 0 preserves the value
a ^ a ^ b       # b, xor-ing twice with the same value cancels out
```

NOT Properties

- Python integers have arbitrary precision (there is no fixed width), so `~a` cannot flip *all* bits; it is defined as $\sim a = -(a+1)$ instead.
- If we want a fixed-width bit-flipped version, we can mask the result:

```
def not_8bit(a):
    return ~a & 0xFF # 0xFF = 11111111, keeps only last 8 digits
```

Shifting

```
1 << n          # 2^n
x >> n          # floor(x / 2^n)
```

13.2 Common Algorithms

Check if the i-th bit is set

```
def is_bit_set(n,i):
    return (n>>i) & 1 == 1
```

Set the i-th bit

```
def set_bit(n,i):
    return n | (1 << i)
```

Clear the i-th bit

```
def clear_bit(n,i):
    return n & ~(1 << i)
```

Toggle the i-th bit

```
def toggle_bit(n,i):
    return n ^ (1 << i)
```

Check if a number is a power of 2

```
def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0
```

Count set bits

- `n & (n - 1)` clears the lowest set bit, so the loop runs once per set bit. (This is also why the power-of-two test above works.)

```
def count_bits(n):
    count = 0
    while n:
        n &= n - 1
        count += 1
    return count
```

XOR based pattern problems

```
# every number appears twice except one: find the single one
from typing import List

def findSingle(nums: List[int]) -> int:
    ans = 0
    for x in nums:
        ans ^= x
    return ans
```

```
# find the missing number in a range 0-n
from typing import List

def findMissing(nums: List[int]) -> int:
    result = len(nums)
    for i, num in enumerate(nums):
        result ^= i^num
    return result
```

```
# swapping two numbers without a temporary (in Python, a, b = b, a is the
  ↪ idiomatic way)
a, b = 5, 3
a = a ^ b
b = a ^ b # (a ^ b) ^ b = a
a = a ^ b # (a ^ b) ^ ((a ^ b) ^ b) = b
```

```
# all subsets
from typing import List

def allSubsets(nums: List[int]) -> List[List[int]]:
    n = len(nums)
    subsets = []

    for mask in range(1 << n): # 2^n choices
        subset = [nums[i] for i in range(n) if mask & (1 << i)]
        subsets.append(subset)

    return subsets
```